What's new in DeployHub

Everything delivered on 26 October 2026: real deployments, environment variables, fixed addresses, routing, one-click databases, public links and Forgot password

> **In short:** DeployHub now really deploys. It fetches a public repository, builds it with Docker, runs it in a locked-down container on this Mac and only calls it LIVE once the app answers. Each project gets a fixed address that survives redeploys, can receive encrypted settings such as database passwords, and can send a path like **/api** to another project, so a frontend and its backend work together without CORS. Apps get a MySQL database in one click and, if you switch it on, a public https link. Users who forget their password can now reset it with a code sent to their email.

Newest: Forgot password with an email code

The login page now has a **Forgot password?** link. It works in two steps:

Step	What happens
1. Enter your email	DeployHub emails a **6-digit code** (through Gmail). The page always says "if this email has an account, a code is on its way", so nobody can use it to find out who has an account.
2. Enter the code and a new password	If the code is right, the password is changed and you log in with the new one. Failed logins from before the reset no longer block you.

- **Safe codes:** only a scrambled (BCrypt) copy is stored. A code works **once**, for **10 minutes**, with **5 guesses**; asking again sends a new code and the old one stops working.
- **No flooding:** at most 5 codes per email address and 20 per network per hour; too many wrong codes from one network are blocked for an hour.
- **To send real emails**, put two lines in `deployhub-backend/.env`: `MAIL_USERNAME=your.address@gmail.com` and `MAIL_PASSWORD=<16-letter App password>`. The App password is made at myaccount.google.com → Security → 2-Step Verification → App passwords (not your normal Gmail password). Restart the backend afterwards.
- **Without those lines** no email is sent: the code appears in the backend's log instead (handy for testing on this Mac).
- New table `PASSWORD_RESET_CODES` (schema version V9); every request, wrong code and reset is written to the audit log.

1. Real deployments (Milestone 6)

Pressing Deploy now goes through four real steps:

Step	What happens
Get the code	A read-only, shallow `git fetch` of the branch, tag or commit from a public repository. The exact commit ID is recorded.
Build	`docker build` with a Dockerfile DeployHub writes for the app type (Java/Spring Boot, Node.js, static site, Python), or with the repository's own Dockerfile. The build command runs inside Docker, never directly on your Mac.
Run	`docker run` with limits (512 MB memory, 1 CPU, 256 processes), all extra Linux privileges removed, as an unprivileged user, reachable only from this Mac.
Health check	The app must answer HTTP on its health check path (default /). Only then is it LIVE, and the replaced version's container is stopped.

- **Your GitHub login is never used.** git runs with saved credentials switched off (macOS keychain, VS Code's helper), so a private repository fails with "Repository not found or not public" instead.
- **Nothing outside your Mac is changed:** no pushes, no webhooks, no settings on GitHub or GitLab.
- **Cancel stops a running build within about 2 seconds.** Long builds keep their claim on the job through a background heartbeat.
- **Rollbacks are exact and fast:** they redeploy the recorded commit and reuse the stored image when nothing changed. Images of the last 3 successful deployments per project are kept.
- **Clear errors:** a missing branch, a failing build command or a crashing app fail at once with the reason and the app's last output. Docker not running or network trouble is retried automatically.
- **The simulator still exists** (`deployhub.execution.mode=SIMULATED`) for the automated tests and for running without Docker.
- **New project settings:** the app's **port** (default per type: Java 8080, Node/static 3000, Python 8000) and **health check path**. The start command is optional for Dockerfile projects.

2. Fixed address per project

Each project gets one address, for example `http://localhost:19001`, at its first real deployment. It never changes: redeploys and rollbacks just switch it to the new app, without downtime.

- A small proxy container (nginx, 64 MB) holds the address and forwards to the project's live app.
- **Open app** on the project and deployment pages uses this address.
- With no live app, the address answers "No live deployment of project ..." instead of an error page.

3. Routes: frontend and backend together

A project can send a path of its address to another project, for example **/api** to your backend:

Request to the frontend's address	Goes to
`http://localhost:19002/`	the frontend project's app (your Angular files)
`http://localhost:19002/api/employees`	the backend project's app (your Express API)

- Your frontend calls `/api/...`: no backend URL in its code, and no CORS setup, because to the browser everything comes from one address.
- When the backend is redeployed, every project that routes to it follows automatically.
- Set routes on the project page (card **Routes**); they apply at once.

4. Environment variables

Settings such as `DB_HOST`, `DB_PASSWORD` or `JWT_SECRET` are given to your app when it starts (card **Environment variables** on the project page).

- **Encrypted** (AES-256-GCM) before they are stored; the key lives in your git-ignored `.env` as `DEPLOYHUB_SECRETS_KEY`. Never change or lose it.
- **Write-only:** after saving, only the name is ever shown. To change a value, type a new one.
- **Kept out of sight:** passed to Docker by name only (never on a command line) and replaced by *** if the build or the app prints them.
- They apply at the next deployment. `PORT` is set by DeployHub and can't be overridden.

5. One-click databases

On a project's page, **Add MySQL database** creates a database just for that project and fills in its settings (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DATABASE_URL`) as encrypted environment variables.

- All projects share one MySQL 8.4 server (`deployhub-mysql`), started on first use; each project's user can only open its own database.
- The password is shown once; **Reset password** makes a new one and updates the settings.
- From this Mac you can connect at `127.0.0.1:3307` (e.g. to create tables). Details in database.pdf.

6. Public links

- Switch on **Public link** on a project and it gets an https address anyone on the internet can open (a free Cloudflare quick tunnel; no account needed). **Open app** then opens that link.
- The link works while this Mac and Docker are running, and it changes when the tunnel restarts; DeployHub checks it every 30 seconds and always shows the current one.
- Make the frontend public and keep the backend private: it is reached through the frontend's `/api` route.

7. Smaller improvements

- **Delete deployments:** failed, cancelled and replaced ones can be deleted (button on the deployment page, trash icon in the list), with their logs and history. The live one and ones in progress can't.
- **Apps can write to their own folder** (e.g. `uploads/`). Those files are lost on redeploy, as on other platforms.
- **Clearer labels:** deployments show "Docker" or "Simulated", the exact commit, and the app's address. The project button is now called **Edit project** (not to be confused with account Settings).
- **Clearer build errors:** a failed build shows the real error line, not Docker's summary or a Docker Desktop link.

8. Earlier today: Milestone 5 and fixes

Feature	In short
Live logs	Lines appear as they are written (Server-Sent Events); after a dropped connection the browser continues where it stopped. Falls back to polling.
Metrics	Queue depth, wait time, throughput, retries and durations for Prometheus, plus a "Last 60 minutes" panel on the dashboard.
Health checks	Liveness (is it running?) and readiness (database and worker OK?); graceful shutdown.
Rollback	Redeploy an earlier deployment that was live, as a new deployment.
Fixes	15 bugs found and fixed in total; see bugreport.pdf for every one.

Tests

Suite	Start of today	Now
Backend on H2 (quick, in-memory database)	112	**232**
Backend on real Oracle (Docker)	72	**136**
Real Docker: git, build, run, proxy, routing, MySQL	0	**7**
Frontend	70	**105**

How to deploy an app with a frontend, a backend and a database

Step	What to do
1. Database	On the backend project, click **Add MySQL database**. The DB_* settings are filled in for you.
2. Backend project	Type Node.js (or your stack). If the code is in a subfolder: build `cd backend && npm ci`, start `cd backend && npm start`. Add other environment variables (e.g. `JWT_SECRET`). Deploy.
3. Frontend project	Type Static site. In your code, call the API at `/api`. Add the route /api -> backend project. Deploy, switch on Public link if others should see it, and click Open app.

Still limited

- Apps run on this Mac: public links work only while it is on, and change when the tunnel restarts. A server in the cloud with fixed https addresses is Milestone 7.
- No backups of Oracle or MySQL yet.
- A password reset doesn't log out sessions already open on other devices (they end after 30 idle minutes). Gmail sends about 500 emails a day at most.
- Public repositories only; no build on push (webhooks would change things on GitHub).
- Files an app writes are lost on redeploy; a LIVE app isn't monitored after its health check.
- Environment variables are available when the app runs, not during its build.
- Docker has about 4 GB here, shared with Oracle: keep only a few apps live at once.

Details: README.md · decision records docs/adr/0006 to 0009 · databases: docs/database.pdf · libraries: docs/third-party-libraries.pdf · bugs: docs/bugreport.pdf